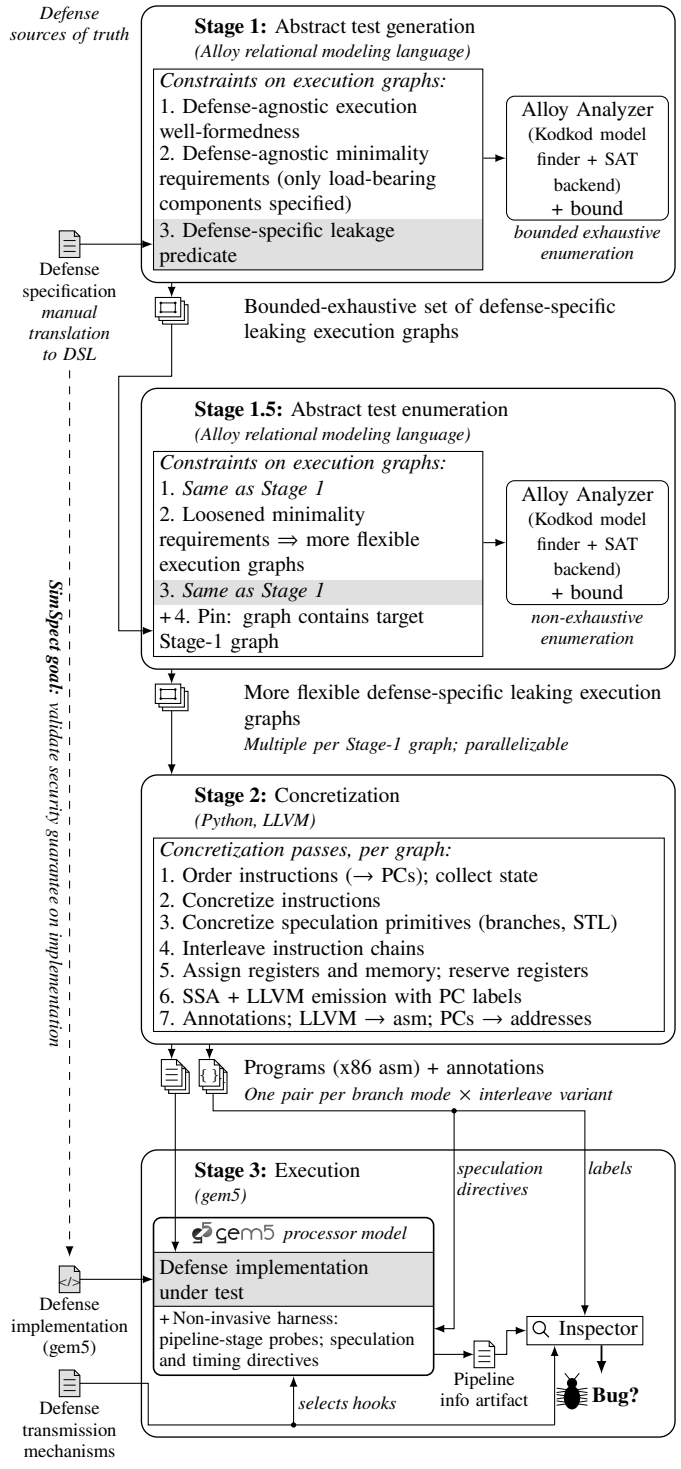

Defense sources of truth
Stage 1: Abstract test generation
(Alloy relational modeling language)
Constraints on execution graphs:
1. Defense-agnostic execution well-formedness
2. Defense-agnostic minimality requirements (only load-bearing components specified)
3. Defense-specific leakage predicate
Alloy Analyzer (Kodkod model finder + SAT backend) + bound
bounded exhaustive enumeration
Defense specification
manual translation to DSL
Bounded-exhaustive set of defense-specific leaking execution graphs
Stage 1.5: Abstract test enumeration
(Alloy relational modeling language)
Constraints on execution graphs:
1. Same as Stage 1
2. Loosened minimality requirements ⇒ more flexible execution graphs
3. Same as Stage 1
+4. Pin: graph contains target Stage-1 graph
Alloy Analyzer (Kodkod model finder + SAT backend) + bound
non-exhaustive enumeration
SynSpec goal: validate security guarantee on implementation
More flexible defense-specific leaking execution graphs
Multiple per Stage-1 graph; parallelizable
Stage 2: Concretization
(Python, LLVM)
Concretization passes, per graph:
1. Order instructions (→ PCs); collect state
2. Concretize instructions
3. Concretize speculation primitives (branches, STL)
4. Interleave instruction chains
5. Assign registers and memory; reserve registers
6. SSA + LLVM emission with PC labels
7. Annotations; LLVM → asm; PCs → addresses
Programs (x86 asm) + annotations
One pair per branch mode × interleave variant
Stage 3: Execution
(gem5)
speculation directives
labels
gem5 processor model
Defense implementation under test
+Non-invasive harness: pipeline-stage probes; speculation and timing directives
Inspector
Pipeline info artifact
Bug?
selects hooks
Defense implementation (gem5)
Defense transmission mechanisms